



Eötvös Loránd University

Faculty of Informatics

THESIS

To obtain Master degree in Computer Science

Specialization: Cybersecurity

**Federated Learning with Moving Target
Defense: Attack Analysis and Defense
Optimization**

Prepared by:

HUSNAIN ALI (BCYW9R)

Under the supervision of:

AYA KHEDDA

Academic Year: 2025-2026

Acknowledgements

I am forever grateful to my supervisor, Aya Khedda, for her constant support, astute advice, and professional guidance during the course of this research. Her feedback and advice played a crucial role in making this thesis a success.

I also thank the faculty members in the Computer Science Department for the challenging academic atmosphere and the facilities necessary to carry out this research.

I am also grateful to my colleagues and friends for their useful discussions and moral encouragement which have been of great help for this work. Finally, I am much obliged to my family for their patience, understanding and constant encouragement during my academic journey.

Husnain Ali

Abstract

What happens when the very clients trusted to train a machine learning model secretly work to destroy it? This is the reality of poisoning attacks in Federated Learning, a privacy-enhancing approach in which a server aggregates clients' updates to a global machine learning model without access to their local data. Although Federated Learning ensures data privacy, it also introduces a fundamental vulnerability: the server cannot determine whether the updates it receives are honest or malicious. This work explores this vulnerability by examining how poisoning attacks impact federated learning systems and to what extent existing privacy mechanisms, such as secure aggregation using Multi-Party Computation, can inadvertently amplify these threats. Furthermore, this thesis proposes a Moving Target Defense configuration that combines secure aggregation with trust-based scoring, permanent identification and removal of malicious clients, dynamic client rotation, and adaptive client selection. The proposed approach aims to simultaneously preserve privacy and robustness, defending against poisoning attacks without requiring the server to inspect any individual client updates.

Keywords: Federated Learning, Moving Target Defense, Multi-Party Computation, Poisoning Attack, Secure Aggregation, Sign-Flipping, Trust Scoring.

Contents

List of Figures	V
List of Tables	V
List of Acronyms	VI
Introduction	1
1 Literature review	4
1.1 Federated Learning	4
1.2 Attacks on Federated Learning	5
1.2.1 Privacy Attacks	5
1.2.2 Poisoning Attacks	5
1.2.3 Backdoor Attacks	6
1.2.4 Sybil Attacks	6
1.2.5 Free-Rider Attacks	6
1.3 Byzantine-Robust Aggregation	7
1.4 Moving Target Defense in Federated Learning	7
1.5 Secure Aggregation and Multi-Party Computation	8
1.6 Summary	9
2 Methodology	10
2.1 Introduction	10
2.2 MPC-based FL Configuration	10
2.2.1 Additive Secret Sharing	11
2.2.2 Client Workflow	11
2.2.3 MPC Nodes	11
2.2.4 Server Aggregation	11
2.3 MTD Configuration (Proposed Defense)	12
2.3.1 Client Pool and Selection	13
2.3.2 Trust Scoring System	13
2.3.3 Detection Mechanism	13
2.3.4 Banning Mechanism	14
2.3.5 Adaptive Node Selection	14
2.3.6 Warmup Period	15
2.3.7 Recovery Mechanism	15
2.4 Attack Model	15
2.4.1 Attack Mechanism	15

2.4.2	Malicious Client Configuration	15
2.5	Experimental Setup	16
2.5.1	Hardware and Software Environment	16
2.5.2	Dataset	16
2.5.3	Model Architecture and Training Configuration	17
2.5.4	Federated Learning Setup	18
2.5.5	Evaluation Metrics	18
2.6	Summary	19
3	Analysis and Evaluation	20
3.1	Introduction	20
3.2	Results	20
3.2.1	Accuracy	20
3.2.2	Loss	22
3.2.3	Latency	23
3.2.4	Secure Aggregation Overhead	23
3.2.5	MTD-Results	24
3.2.6	Stability Analysis	25
3.2.7	Computational Overhead Comparison	26
3.2.8	Summary of Comparative Findings	26
3.3	Discussion	27
3.3.1	Impact of sign-flipping on Baseline and MPC-based FL	27
3.3.2	Impact of sign-flipping on MTD-based FL	27
3.3.3	Trade-offs and Limitations	28
3.3.4	Comparison with Related Works	29
3.3.5	Implications for Federated Learning Security	29
3.4	Summary	30
4	Conclusions and Perspectives	31
4.1	Summary of the Work	31
4.2	Limitations	31
4.3	Conclusions	32
4.4	Future Perspectives	32
4.5	Code Availability	33
A	Implementation Code Snippets	36
A.1	Introduction	36
A.2	Model Architecture	36
A.3	Sign-Flipping Attack	36
A.4	Additive Secret Sharing	37
A.5	MTD Trust Score Computation	38
A.6	MTD Banning Mechanism	38
A.7	MTD Adaptive Node Selection	38

List of Figures

2.1	Complete Architecture	10
3.1	Accuracy Comparison of Baseline vs MPC vs MTD	20
3.2	Loss Comparison: Baseline vs MPC vs MTD	22

List of Tables

2.1	Detection Conditions	14
2.2	Adaptive Node Selection	14
2.3	Distribution of Malicious Clients	16
2.4	Hardware Specifications	16
2.5	Configuration Settings	17
3.1	Final Accuracy Comparison (Round 200)	21
3.2	Final Loss Comparison (Round 200)	22
3.3	Baseline, MPC & MTD Round Time Statistics	23
3.4	Secure Aggregation Overhead: MPC vs MTD	23
3.5	Trust Score Evolution (All Clients)	24
3.6	Helper Node Usage Statistics	25
3.7	Performance Stability Metrics (Rounds 100–200)	25
3.8	Computational Overhead Comparison	26
3.9	Overall Comparison Summary	26
3.10	MTD Trust Parameters	28
3.11	Comparison with Related Work	29

List of Acronyms

AI	Artificial Intelligence
CAGR	Compound Annual Growth Rate
CNN	Convolutional Neural Network
DLG	Data Leakage from Gradients
FedAvg	Federated Averaging
FEMNIST	Federated Extended MNIST
FL	Federated Learning
IID	Independent and Identically Distributed
MEC	Mobile Edge Computing
MLP	Multi-Layer Perceptron
MNIST	Modified National Institute of Standards and Technology
MPC	Multi-Party Computation
MTD	Moving Target Defense
ReLU	Rectified Linear Unit
SGD	Stochastic Gradient Descent

Introduction

Federated Learning (FL) is a revolutionary approach to machine learning that allows multiple clients to train a shared model without exchanging their data. This privacy-preserving technique has seen rapid adoption in privacy-sensitive applications, including healthcare, finance, and mobile apps. Recent market analysis shows the global federated learning market stood at USD 138.6 million in 2024 and is expected to reach USD 297.5 million by 2030, with a compound annual growth rate (CAGR) of 14.4%, driven by the growing demand for privacy-preserving AI applications in healthcare, finance, and telecommunications [9].

Despite these privacy advantages, Federated Learning is not inherently secure and remains vulnerable to a range of attacks that can undermine the integrity of the shared model. These security challenges become especially critical in such sensitive deployments, where the consequences of a compromised model can be severe. Addressing these vulnerabilities is therefore essential to ensuring that federated learning systems remain both privacy-preserving and trustworthy across sensitive real-world applications.

Problem Statement

The centralized architecture of Federated Learning, necessary to preserve privacy, introduces a trust issue. The server aggregates model updates from all participating clients without the ability to verify their correctness, leaving it unaware of each client's local training procedure and unable to confirm whether updates are honest or malicious. This vulnerability allows for poisoning attacks, where malicious clients can alter their model updates to negatively impact the global model's accuracy. The most insidious type of poisoning attack is the sign-flipping attack, which flips the sign of a client's gradient update and scales it up, causing the global model to diverge instead of converge.

This issue is made even more challenging by the deployment of secure aggregation methods such as Multi-Party Computation (MPC), which prevent the server from examining individual updates. Although MPC offers high levels of privacy, it also hinders the server's ability to identify suspicious or malicious updates, establishing a trade-off between privacy and robustness. This need for a poisoning defense mechanism that simultaneously provides privacy, robustness against poisoning, and permanently removes malicious clients from the training process is the core motivation of this thesis.

Aim and Objectives

A. Aim

The aim of this thesis is to design, implement, and evaluate a Moving Target Defense (MTD) solution for Federated Learning that offers privacy via secure aggregation and resilience to sign-flipping poisoning attacks without requiring the server to inspect individual client updates.

B. Objectives

1. Establish a baseline for attack impact measurement

Implement a standard Federated Averaging (FedAvg) configuration with no defense mechanisms and evaluate its robustness against an amplified sign-flipping attack.

2. Design and implement an MTD-based defense

Design and implement an innovative defense configuration using MPC-based secure aggregation with trust-based scoring of clients, banning of malicious clients, dynamic client selection, and adaptive rotation of helper nodes.

3. Evaluate the proposed approach

Evaluate the effectiveness of the proposed MTD in terms of performance and robustness against poisoning attacks in comparison with baseline FL and pure MPC-based FL.

Contribution and Methodology

The main contribution of this thesis is the design and experimental analysis of a Moving Target Defense configuration for Federated Learning that provides privacy and robustness. The design of the MTD does not require examining the updates of individual clients and is compatible with secure aggregation.

Three configurations are evaluated: a Baseline (standard FedAvg with no defense), an MPC configuration that achieves privacy through secure aggregation but offers no robustness, and the proposed MTD configuration that extends MPC with trust-based client scoring, permanent banning of malicious clients, dynamic client rotation, adaptive helper node selection, and a warmup period to avoid false suspicion during early training.

The experimental results demonstrate that the proposed MTD configuration effectively mitigates the sign-flipping attack, achieving a final accuracy of 96.44%, compared to 50.67% for the Baseline and 45.74% for the MPC configuration. The MTD mechanism successfully detected and permanently banned 5 out of 6 malicious clients (83%) without banning any honest client, with a secure aggregation overhead of just 0.1% of each round's duration.

Thesis Structure

- **Chapter 1 - Literature review:** Provides background on Federated Learning, attacks in FL including poisoning attacks, Byzantine-robust aggregation methods, Moving Target Defense in federated learning, and secure aggregation using Multi-Party Computation. This chapter positions our work with respect to prior research.
- **Chapter 2 - Methodology:** Provides detailed descriptions of the MPC and MTD experimental configurations, including additive secret sharing, client protocol, server aggregation, trust scoring, banning, and dynamic node selection. It also details the attack model, experimental protocol, dataset, model architecture, training hyper-parameters, and evaluation metrics.
- **Chapter 3 - Analysis and Evaluation:** Evaluates and discusses the experimental results per configuration, followed by a comparative analysis of accuracy, loss, round time, computational overhead, trust score dynamics, banning, and suspicion events. The chapter concludes with an analysis of the results and a discussion of the security of federated learning.
- **Chapter 4 - Conclusions and Perspectives:** This chapter presents the main findings and conclusion of the thesis, discusses limitations, and proposes future research directions.

Chapter 1

Literature review

This chapter provides background on federated learning and its security vulnerabilities. Federated Learning (FL) is a distributed machine learning paradigm that allows clients to jointly learn a model without sharing their data. This approach provides benefits in privacy, but it also comes with security risks. Particularly concerning are poisoning attacks, in which malicious clients modify their local models to adversely affect the global model. Defending against such attacks requires approaches ranging from Byzantine-robust aggregation to secure aggregation via multi-party computation and proactive strategies like Moving Target Defense.

1.1 Federated Learning

Federated Learning (FL) is a distributed machine learning framework introduced by McMahan et al. [14], in which clients collaboratively train a shared machine learning model without sharing their local data with a server. Local data is retained by clients, while model updates are shared with the server to maintain privacy. Training is performed in rounds using an algorithm called Federated Averaging (FedAvg) [14].

In each round, the server distributes the current global model to selected clients. Each client trains the model on its local data using an optimization algorithm such as Stochastic Gradient Descent (SGD) for a few local epochs. The clients send their updated models to the server. The server then updates the global model with the weighted average of all the local models. This process is repeated for a fixed number of rounds or until the model reaches acceptable performance. In this model, the server doesn't see any data from the clients; all it does is facilitate the model training by sending models to clients and aggregating model updates.

The server-coordinated approach is the standard architecture for production FL systems. Bonawitz et al. [6] outlined the system design and engineering challenges for scaling FL in a mobile keyboard application at Google, many of which serve as the basis for common design patterns seen in later FL frameworks. This design has been adopted for production FL deployments such as Google's Gboard for next-word prediction, Apple's Siri for speech recognition, and WeBank for credit risk assessment.

FL deployments are often classified into two settings [18]. Cross-device settings involve thousands to billions of clients, with only a fraction of clients participating in each round. Cross-silo settings involve a smaller number of clients (up to a hundred) and all or most of the clients participate in each round. Cross-silo settings are common in collaborative learning scenarios across multiple institutions, such as hospitals, banks or laboratories, where each client is an entity representing the institution, rather than a device.

1.2 Attacks on Federated Learning

FL's decentralized approach introduces a key security challenge: clients train models locally and send updates to a server without the server controlling the training process, opening up several avenues for attack. In contrast to traditional machine learning, where training is under the control of a trusted central authority, FL is decentralized and control is spread among many clients, some of which may be malicious. This design feature enables a range of attacks, which can be broadly grouped into privacy attacks, poisoning attacks, backdoor attacks, Sybil attacks, and free-rider attacks.

1.2.1 Privacy Attacks

Privacy attacks seek to steal information about clients' private data from the model updates. These attacks rely on gradients and the model's parameters memorising training data. Privacy attacks are further divided into:

DLG (Data Leakage from Gradients): Adversaries recover training data from gradients by solving an optimization problem that finds the gradient of dummy data that matches the observed gradient. This attack showed that even sharing gradients, which are typically assumed to be safe, can lead to leaked training data.

Inference Attacks: These attacks infer characteristics of the training data without directly recovering it. They are further divided into:

- **Membership Inference:** Reveals whether a particular data sample has been included in the training data. This can disclose private information, like whether a specific patient's data was used to train a medical dataset.
- **Property Inference:** Infers statistical properties of the training data distribution, such as the percentage of a specific population group, without revealing individual information.

Model Inversion: Reconstructs training data samples from the global model. This can be a serious threat in applications such as face recognition or medical imaging, where the generated images can reveal personal details.

1.2.2 Poisoning Attacks

Poisoning attacks seek to reduce the model's performance or inject malicious functionality by manipulating the training data or model updates [8, 17]. These attacks exploit the

trust FL has in clients. Shejwalkar et al. [18] offer a detailed taxonomy of poisoning attacks, classifying them based on three criteria: adversary goal (targeted, backdoor, or untargeted), adversary knowledge (white-box or no-box), and adversary capability (data poisoning or model poisoning). There are two types of poisoning attacks:

Data Poisoning: Adversaries tamper with their training data. This attack is usually simpler because adversaries only have to modify their own training data, but its effects on the global model are secondary and can be masked by non-adversaries.

Model Poisoning: Adversaries manipulate the model updates before sending them to the server [2, 20]. This attack is more effective than data poisoning because the adversaries can generate malicious updates that have the largest possible impact on the global model. Model poisoning is further divided into:

- **Local Model Poisoning:** One adversary poisons its own update. This is less effective but easier to perform and potentially harder to detect.
- **Global Model Poisoning:** Several adversaries collaborate on their malicious updates to have a greater impact on the global model.

1.2.3 Backdoor Attacks

Backdoor attacks are a type of poisoning attack in which adversaries inject backdoors into the model. The model's behavior is unaffected by normal inputs, making backdoor attacks hard to detect in standard testing, but it gives a desired output when a trigger pattern is present in the input. For instance, in an image classifier, all traffic signs may be correctly classified, but stop signs with a small yellow sticker may be mistakenly classified as a speed limit sign. Backdoor attacks are especially dangerous because they are triggered by the trigger.

1.2.4 Sybil Attacks

Sybil attacks take advantage of the inability of FL servers to identify multiple clients controlled by the same attacker. An attacker can generate multiple fake client identities, each sending malicious updates to the FL server, thus increasing their impact on the global model beyond their computational power. This attack boosts the adversary's voting rights in the aggregation process without them needing to increase their computational resource allocation.

1.2.5 Free-Rider Attacks

Free-rider attacks are another attack where the goal is to obtain the global model without providing valuable training. Free-riders provide random, zero, or weak updates to get the updated global model without legitimate training costs. This attack does not affect the model performance directly, but it undermines the collaborative nature of FL and slows down the convergence, while also putting a disproportionate burden on honest clients who carry the computational costs.

1.3 Byzantine-Robust Aggregation

The initial line of defense examined in the FL literature is to substitute the simple averaging step of FedAvg with a more robust aggregation rule that is less vulnerable to outliers and adversarial updates. One of the first Byzantine-tolerant distributed learning aggregation rules was proposed by Blanchard et al. [4] as Krum. Krum calculates pairwise distances among all the submitted updates in every round and chooses the single update that has the smallest total squared distances to its nearest neighbors, on the assumption that honest updates are concentrated together and malicious updates are outliers. It has a form of variation known as Multi-Krum, which averages the top-k nearest updates rather than picking a single one and offers a more continuous aggregation.

Yin et al. [21] proposed coordinate-wise Trimmed Mean and Median aggregators. In every parameter dimension, Trimmed Mean eliminates the biggest and smallest values and averages the remaining values. Median simply takes the per-dimension median. Both offer convergence guarantees in theory under bounded fractions of Byzantine workers, and are computationally inexpensive compared to Krum since they run per-dimension, not on full update vectors. Cao et al. [7] adopted a different approach in FLTrust. FLTrust provides the server with a small clean dataset and it computes its own reference update at every round. Client updates are then ranked based on their cosine similarity to the server reference, contributions that are negatively similar are discarded, and contributions that are positively related are weighted by the similarity. This method is effective in transforming client updates into directional votes compared to a trusted reference direction.

These Byzantine-resilient aggregation rules are effective in resisting simple outlier-style attacks, but Fang et al. [8] and Baruch et al. [2] showed that they can be circumvented by adaptive attackers. Also, each of the three classical approaches demands that the server look at individual client changes in cleartext, which is inconsistent with the notion of secure aggregation protocols that seek to obscure individual client updates from the server. This tension between privacy and robustness drives solutions such as the MTD mechanism employed in this thesis, which integrates MPC-based privacy with client-level trust scoring rather than update inspection.

1.4 Moving Target Defense in Federated Learning

Moving Target Defense (MTD) is a proactive security mechanism that originated in network and systems security. The fundamental concept is to keep varying the attack surface of the system such that the adversary's knowledge of the system becomes obsolete. Instead of only detecting an attack after it has happened and reacting, MTD changes system parameters dynamically, making it more expensive and uncertain for the attacker.

The concept of reputation-based client filtering, which is also associated with MTD, has been addressed in the FL context as well. Sharma, Chen, Zhao, Qiu, Bagchi and Chaterji [16] suggested FLAIR, a defense against what they term a directed deviation attack, which is essentially a sign-flipping attack in which malicious clients flip the sign of the gradient updates to misdirect the model out of convergence. FLAIR calculates a flip-score for each client measuring the extent to which the gradient signs of that client go

against the majority direction. Clients with continually high flip-scores round after round obtain lower reputation weights and thus have less impact on the aggregate model. FLAIR tested on MNIST and FEMNIST with 80 benign and 20 malicious clients demonstrated robustness at 20–30% attacker ratios.

Kianpisheh, Benzaid, and Taleb [11] applied the concept of MTD more directly to federated learning by introducing a multi-model framework, where the client models evolve dynamically with training epochs. Their approach views model architectural diversity as a kind of topology mutation: as the model structures involved in each round change, the model poisoning attack surface changes, and it becomes more difficult to design successful poisoned updates that fit the desired model format. They tested this in a Mobile Edge Computing (MEC) environment and showed greater resistance to model poisoning attacks than in the case of static single-model FL.

Kianpisheh and Taleb [12], in a later paper, explicitly generalized this work as a topology mutation strategy, calling it Moving Target Defence. Their MTD-FL system actively selects the devices that join the federation in real time and relies on traffic-level measurement and expected Byzantine status, and the topology mutation is optimized by deep reinforcement learning. In poisoning attacks, their strategy eliminated 87%–96% of malicious models during aggregation and significantly minimized the effect of poisoned updates on the global model.

The MTD mechanism of the current thesis is based on the same concept of dynamic participant management present in FLAIR [16] and Kianpisheh et al. [11, 12]. FLAIR dynamically changes client reputation weights using flip-scores to minimise the impact of malicious clients over time. Kianpisheh et al. dynamically alter the model architecture in each round, thereby changing the attack surface and making poisoning harder. Both use the fundamental MTD idea of dynamic, rather than static, defence.

1.5 Secure Aggregation and Multi-Party Computation

In FL, the training data is kept locally on the clients, but the model updates can reveal information about the data. Secure aggregation prevents this by allowing the server to compute the aggregate of the client updates while hiding each client update from the server.

The first practical approach to secure aggregation in FL, proposed by Bonawitz et al. [5], is based on pairwise masking and secret sharing. In their protocol, every pair of clients shares a random mask that cancels when summed, and Shamir secret sharing is used to deal with client dropouts. In this protocol, the server learns only the sum of clients' updates, not the individual updates, and clients can drop out in some proportion during each round. The Bonawitz protocol recognised secure aggregation as one of the building blocks of privacy-preserving FL.

A multi-party FL (MPCFL) algorithm introduced by Alawadi et al. [1] is based on additive secret sharing and multi-party computation (MPC). In their protocol, clients split their model update into shares and send them to multiple computation nodes. The model update is then aggregated on the shares without reconstructing any individual client's

update, so the server or any individual computation node does not know the update of any client. They evaluated MPCFL on several benchmark datasets, showing that it can reach the same model accuracy as the baseline FedAvg, while offering privacy protection against inference attacks and gradient leakage.

This privacy-robustness trade-off is a research challenge in FL. Secure aggregation keeps individual updates hidden from the server, which also means the server cannot check the updates for any anomalies. Traditional Byzantine-robust aggregators (Krum [4], Trimmed Mean [21], FLTrust [7]) require access to the individual updates, and are thus incompatible with secure aggregation.

This problem has been tackled recently. Mai, Yan and Pang [13] introduced RFLPA, a robust FL protocol that uses secure aggregation for poisoning defense by calculating cosine similarity between client and server updates within an MPC protocol. They use verifiable packed Shamir secret sharing and a new dot-product aggregation protocol to achieve reduced communication overhead while preserving privacy and robustness.

1.6 Summary

In this chapter, we cover the background on federated learning (FL) and security challenges. FL allows for distributed training without sharing data, but this decentralization might result in vulnerabilities. There are many attacks against FL systems including privacy attacks, poisoning attacks, backdoor attacks, etc., which can be mitigated using techniques such as Byzantine-robust aggregation (e.g., Krum, Trimmed Mean), Moving Target Defense, and secure aggregation of client updates. However, there is a trade-off between privacy (hiding individual updates from the server) and robustness (needing to inspect updates for malicious behavior). Addressing this trade-off is the central goal of this thesis work which is presented in the next chapters.

Chapter 2

Methodology

2.1 Introduction

This chapter describes the experimental setup used in evaluating federated learning setups against the sign-flipping poisoning attack. The chapter introduces the system and software environment, the common configurations for the experiment such as model architecture, dataset and training hyperparameters, and the attack model. It then describes the federated learning configurations and their evaluation metrics and procedure.

2.2 MPC-based FL Configuration

The MPC (Multi-Party Computation) setup builds on the baseline FL with secure aggregation via additive secret sharing. This setup uses three helper nodes (i.e, MPC nodes) in addition to the server and the clients. These nodes are static and participate in the secure aggregation protocol as depicted in Figure 2.1

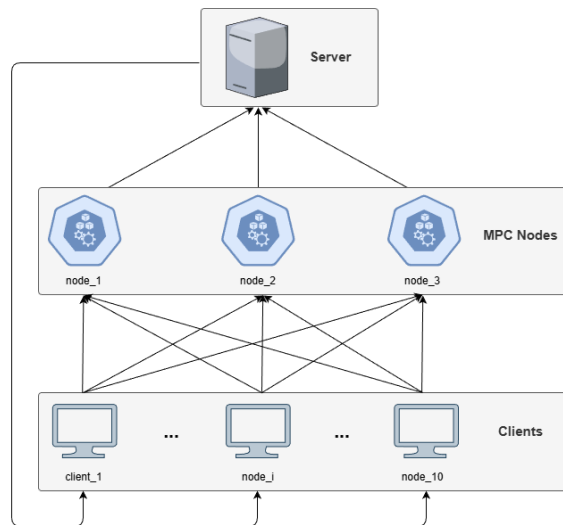


Figure 2.1: Complete Architecture

2.2.1 Additive Secret Sharing

The MPC setup implements Additive Secret Sharing, where a given client's model weights are represented as a vector w of dimension d is split into shares denoted as s as follows:

$$w = s_{server} + s_{node1} + s_{node2} + s_{node3}$$

where each s_{node} share is a vector of d random value drawn from a normal distribution $\mathcal{N}(0, 1)$ and the server's share is calculated as follows:

$$s_{server} = w - (s_{node1} + s_{node2} + s_{node3})$$

2.2.2 Client Workflow

Every client in the MPC setup will go through the following steps each round:

1. Receive the global model from the Flower server.
2. Complete local training for one epoch.
3. Apply the sign-flipping attack if malicious.
4. Split the weights into shares.
5. Send the shares to the server and the MPC nodes.

2.2.3 MPC Nodes

In the pure MPC-based protocol, each helper node receives shares for the model update from all clients. The node keeps a cumulative sum of the shares for each round. It holds the sum until all shares have been collected and then responds to the server's request. This approach guarantees that the server does not receive any individual client's update, therefore maintaining the privacy of the MPC protocol.

2.2.4 Server Aggregation

The server uses a custom aggregation strategy FedAvgWithASS. For each round:

1. Waits until it receives the shares from all 10 clients.
2. Sums the clients' shares.
3. Requests and receives node sums from all three MPC nodes.
4. Reconstructs the global model and sends it to the clients.

The server never receives full client weights, thereby maintaining privacy through the additive secret sharing.

2.3 MTD Configuration (Proposed Defense)

The main contribution of this thesis is the MTD (Moving Target Defense) configuration. It builds upon the MPC configuration (see section 2.2) by introducing trust-based client scoring, permanent banning of malicious clients, dynamic client selection, and MPC node rotation. The workflow of the MTD server during every round is summarized in the following algorithm (see Algorithm 1).

Algorithm 1: MTD Server Workflow per Round

Input: Global model θ , client_manager, current_risk_state
Output: Updated global model θ' , updated_risk_state

```
1 if server_round = 1 then
2   | Wait for all 20 clients to connect
3 if current_risk_state then
4   | selected_clients  $\leftarrow$  TrustBasedSample(active_clients, 10)
5 else
6   | selected_clients  $\leftarrow$  RandomSample(active_clients, 10)
7 active_nodes  $\leftarrow$  RandomSelect(node_pool,
   compute_active_count(current_risk_state))
8 Distribute global model and active_nodes to selected_clients
9 Collect and aggregate shares (same as MPC)
10 Evaluate global model  $\rightarrow$  loss, accuracy
11 if server_round > WARMUP_ROUNDS then
12   | if loss > 2.0 or accuracy < 0.80 or accuracy < previous_accuracy then
13     | current_risk_state  $\leftarrow$  True
14     | stable_counter  $\leftarrow$  0
15     | for each client in selected_clients do
16       | client.suspicious_count += 1
17       | client.trust_score = recompute_trust(client)
18       | if client.suspicious_count  $\geq$  10 and client.trust_score < 0.6 then
19         | | banned_clients.add(client)
20   | else
21     | if current_risk_state then
22       | stable_counter += 1
23       | if stable_counter  $\geq$  3 then
24         | | current_risk_state  $\leftarrow$  False
25     | for each client in selected_clients do
26       | | client.trust_score = recompute_trust(client)
27 return  $\theta'$ , current_risk_state
```

2.3.1 Client Pool and Selection

The MTD setup has a larger client pool of 20 clients (cid 0-19). Each round, only 10 clients are selected to participate, which provides a rotation surface that minimizes the effects of malicious clients.

Client selection operates in two modes:

- **Safe Mode** (default): Randomly selects active (non-banned) clients only.
- **Risk Mode** (triggered by suspicion): Selects based on the trust of the top 70% most trusted active clients.

The selection mode depends on the detection mechanism as described in Section 2.3.3.

2.3.2 Trust Scoring System

Every client has three trust-related metrics:

- **selected_count**: Number of rounds the client has participated in.
- **suspicious_count**: Number of rounds where the client was considered suspicious.
- **trust_score**: A floating-point number in $[0.1, 1.0]$ that indicates the client's reliability.

Trust Score Calculation:

A suspicion event is assigned to all clients who were part of that round when suspicious global behavior is detected. The updated trust score is:

$$\text{trust} = 1.0 - 0.7 \times \left(\frac{\text{suspicious_count}}{\text{selected_count}} \right) \quad (2.1)$$

$$\text{trust} = \max(0.1, \text{trust}) \quad (2.2)$$

where 0.7 is the trust penalty factor and 0.1 is the minimum trust score.

For example, a client that attended 10 rounds and was suspicious in 3 rounds would have:

$$\text{trust} = 1.0 - 0.7 \times (3/10) = 1.0 - 0.21 = 0.79 \quad (2.3)$$

2.3.3 Detection Mechanism

To detect possible poisoning attacks, the server tracks the global model's behavior at the end of each round. During warmup (first 8 rounds), detection is disabled to allow the model to learn normally.

Suspicion is triggered if any of the following conditions are met after warmup:

Condition	Threshold	Rationale
Loss increase	Loss > 2.0	Unusually high loss indicates model divergence
Accuracy drop	Accuracy < 0.80	Model performing below acceptable threshold
Accuracy decrease	Accuracy < previous round accuracy	Sudden performance degradation

Table 2.1: Detection Conditions

When suspicion is triggered:

- Each participating client’s suspicious_count is incremented by +1.
- Trust scores for all participating clients are recalculated.
- The system enters Risk Mode for the next round.

2.3.4 Banning Mechanism

Clients are ineligible to participate in future rounds if they meet both of the following conditions:

- suspicious_count ≥ 10
- trust_score < 0.6

This dual condition prevents the early exclusion of honest clients who might have incidentally high loss due to normal training dynamics. Once banned, a client is added to the banned_clients set and will not be considered in any further client selection.

2.3.5 Adaptive Node Selection

Mode	Node Ratio	Minimum Nodes	Active Nodes (of 3 total)
Safe Mode	25%	2	2
Risk Mode	75%	3	3

Table 2.2: Adaptive Node Selection

The MTD configuration dynamically adjusts the number of active helper MPC nodes according to the risk state, as illustrated in Table 2.2. The active nodes in each round are chosen randomly from the set of all nodes. This randomization does not allow attackers to target specific nodes across multiple rounds.

2.3.6 Warmup Period

The initial eight rounds are considered warmup rounds, therefore they are run without detection or banning. This allows the model to reach a decent accuracy threshold before the defense mechanisms are activated. All participating clients are treated as honest in trust accounting during warmup.

2.3.7 Recovery Mechanism

The system goes back to Safe Mode after three consecutive rounds with no suspicious activity. This allows the system to reduce the computational load once the threat has been resolved.

2.4 Attack Model

A sign-flipping attack was utilized to test the strength of each setup. This attack is considered an untargeted model poisoning attack, because the attacker aims at compromising the overall model accuracy and not at causing particular misclassifications.

2.4.1 Attack Mechanism

On every malicious client, the attack is applied after local training and before sending the update to the server. Let w denote the local model update of an honest client. A malicious client transforms its update into a poisoned update using a scaled sign-flipping attack defined as follows:

$$w' = -\alpha \times w \quad (2.4)$$

This not only reverses the direction (inverting the direction of the update) but also increases the magnitude by a factor of α . The attack is applied to all model parameters (i.e, including the input-to-hidden layer weights, hidden layer biases, output layer weights, and output layer biases) thereby maximizing its impact on the aggregated global model.

2.4.2 Malicious Client Configuration

In all three configurations, malicious clients had the following:

- Attack type: sign-flipping
- Attack scaling factor: 2.0
- Persistence: Deterministic (applied in each training round in which the malicious client is involved)

The malicious client distribution per configuration is presented in Table 2.3.

Table 2.3: Distribution of Malicious Clients

Configuration	Total Clients	Malicious Clients	Attacker Ratio
Baseline	10	3	30%
MPC	10	3	30%
MTD	20	6	30%

2.5 Experimental Setup

2.5.1 Hardware and Software Environment

The experiments in this work were conducted on a Dell Latitude E5570 laptop. A summary of the hardware specifications is provided in Table 2.4.

Element	Detail
CPU	Intel Core i7 - 6820 HQ @ 2.70 GHz (4 cores, 8 threads)
Memory	16 GB (with 15.9 GB available)
GPU	AMD Radeon R7 M370 (2 GB) + Intel HD Graphics 530
Storage	238 GB SSD (SKhynix SC311 SATA 256GB)
Operating System	Windows 11, 64-bit edition

Table 2.4: Hardware Specifications

The software environment was based on Python (version 3.10) and included the following essential packages:

- **Flower (flwr)** [3]: An open-source federated learning framework for multi-client distributed training. It offers the communication protocol and orchestration logic for server-client communication, including client discovery, round management and model sharing.
- **PyTorch** [15]: An open source machine learning library with tensor computing and automatic differentiation. It was used to define the neural network model and train it locally on clients.
- **TorchVision** [19]: A library that integrates with PyTorch to offer datasets, models and transforms of images. It was used to load and transform the MNIST dataset.
- **NumPy** [10]: A fundamental library for scientific computing with Python that supports large, multi-dimensional arrays and mathematical functions. It was used to manage model parameters and to perform additive secret sharing.

2.5.2 Dataset

The dataset chosen for the model training is the MNIST handwritten digits dataset, which consists of 60,000 training samples and 10,000 testing samples, all represented as 28×28 grayscale images. During preprocessing, the images were converted into tensors, with pixel intensities scaled from the range $[0, 255]$ to $[0, 1]$ without any additional normalization.

Data Segmentation:

In the case of baseline and MPC-based FL, only the initial 30,000 training images were considered. These were divided equally among 10 clients and each client was given 3,000 images with an Independent and Identically Distributed (IID) random split with a fixed seed of 42. However, In the MTD setup, the entire 60,000 training images were utilized, with 20 clients each randomly allocated 3,000 images with the same IID random split with the same seed.

The test set of 10,000 images was kept constant in all configurations and was not shuffled during evaluation.

2.5.3 Model Architecture and Training Configuration

All tests were conducted using a two-layer Multi-Layer Perceptron (MLP) with the following architecture:

Input Layer (784) $\rightarrow$ Hidden Layer (64, ReLU) $\rightarrow$ Output Layer (10)

The MNIST images, which are flattened to 28x28 pixels, have an input dimension of 784. The hidden layer employs ReLU activation, while the output layer produces logits for the 10 digit categories (0-9). The overall count of trainable parameters is 50,890, calculated as:

- Hidden weights input: $784 \times 64 = 50,176$
- Hidden biases: 64
- Weights for hidden output: $64 \times 10 = 640$
- Output biases: 10

Parameter	Value
Optimizer	Stochastic Gradient Descent
Learning rate	0.01
Momentum coefficient	0.9
Mini-batch size	32
Per-client epochs	1

Table 2.5: Configuration Settings

The training hyperparameters presented in Table 2.5 were commonly used across all clients and configurations.

2.5.4 Federated Learning Setup

The federated learning setting was implemented and evaluated using the Flower (flwr) framework. The system follows a centralized, server-coordinated architecture, where clients perform distributed model training and a central server handles global model aggregation. For baseline FL, the server employs Federated Averaging (FedAvg). For MPC-based and MTD-based FL, secure aggregation is performed by 3 MPC nodes rather than the server directly aggregating client updates.

Experiments were conducted with different numbers of participating clients (10 for baseline and MPC-based FL, and 20 for MTD-based FL), 3 MPC nodes, and over 200 communication rounds.

2.5.5 Evaluation Metrics

Each configuration was measured using the following metrics so that a comparative analysis could be conducted.

Accuracy: The global model’s accuracy was assessed after every communication round using the complete MNIST test set (10,000 images). Accuracy is defined as:

$$\text{Accuracy} = \frac{\text{Number of correct predictions}}{\text{Total test samples}} \quad (2.5)$$

Loss: Evaluation loss was computed per sample across all three configurations (Baseline, MPC, and MTD) using cross-entropy loss.

Round Time: Total round time was measured from the start of Flower’s `configure_fit()` to the completion of `aggregate_evaluate()`. This includes client training, communication, and aggregation overhead.

Secure Aggregation Overhead In the case of MPC and MTD setups, the following timing indicators were measured:

- **shares_wait:** Waiting time for all server shares.
- **node_wait:** Time taken to poll all helper node sums.
- **sa_total:** Total secure aggregation time (shares_wait + node_wait (maximum) + reconstruction).

The overhead percentage was calculated as:

$$\text{Overhead (\%)} = \frac{\text{sa_total}}{\text{total_round_time}} \times 100 \quad (2.6)$$

Banning Statistics: In the case of the MTD setup, the following were monitored:

- Number of banned clients per round.
- Cumulative banned clients over 200 rounds.
- Trust scores of banned clients at the time of banning.
- Suspicious event counts of banned clients.

2.6 Summary

This chapter outlined the approach for evaluating federated learning setups against a sign-flipping poisoning attack. The MPC configuration enables privacy-preserving secure aggregation via additive secret sharing with three helper nodes and protects the privacy of individual client updates, however, it does not achieve robustness against poisoning attacks. In contrast, the MTD configuration extends the MPC configuration with additional mechanisms such as trust-based client scoring, permanent removal of malicious clients, dynamic client selection and adaptive MPC node rotation to provide privacy and robustness.

The following chapter presents and analyzes the experimental findings of the two privacy-preserving configurations.

Chapter 3

Analysis and Evaluation

3.1 Introduction

This chapter presents the results of the experiments conducted across the three federated learning configurations under a sign-flipping poisoning attack. The findings reveal how each configuration responds to the attack in terms of model accuracy and stability, and demonstrate the extent to which the proposed Moving Target Defense approach succeeds in preserving both privacy and robustness.

3.2 Results

3.2.1 Accuracy

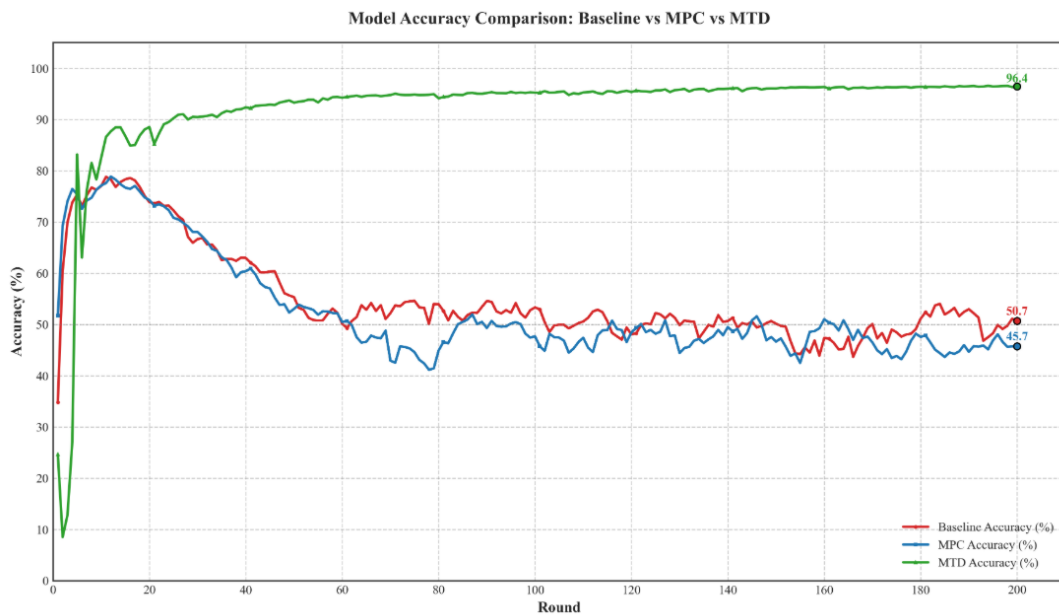


Figure 3.1: Accuracy Comparison of Baseline vs MPC vs MTD

Configuration	Peak Accuracy	Final Accuracy	Accuracy Decline	Recovery
Baseline	78.80% (Round 11)	50.67%	−28.13%	None
MPC	77.10% (Round 10)	45.74%	−31.36%	None
MTD	96.44% (Round 200)	96.44%	0%	Complete

Table 3.1: Final Accuracy Comparison (Round 200)

According to the results shown in Figure 3.1 The Baseline setup exhibits the typical model of a successful poisoning attack. The honest majority (7 out of 10 clients) drives the model towards convergence in the first 11 rounds, achieving 78.8% accuracy. However, the updates of the malicious clients gain power as training continues. Accuracy decreases to 66.65% by Round 30 and to 55.33% by Round 50, which is barely better than random guessing (10% for 10 classes). It drops further to 50.67% by Round 200.

The MPC setup reaches a marginally lower peak accuracy of 77.10% at Round 10, compared to Baseline’s 78.80% at Round 11. Unlike Baseline, which stays above 50% accuracy throughout, MPC accuracy falls below 50% after Round 90 and continues declining to 45.74% by Round 200. This shows that MPC secure aggregation, while providing privacy, can actually make poisoning attacks worse because it faithfully combines all updates without any filtering or anomaly detection.

In contrast, the MTD setup achieves 96.44% accuracy by Round 200, significantly outperforming both Baseline 50.67% and MPC 45.74%. This demonstrates that MTD performs better than either standard or MPC-secured aggregation for defending against persistent poisoning attacks.

The proposed MTD framework operates in three phases:

Phase 1: Warmup (Rounds 1–8)

In the warmup phase, there are no detection or banning actions. The accuracy increases from 24.65% to 81.52% and the loss gets reduced from 2.17 to 0.73 to provide an initial performance baseline before activating the defense mechanisms.

Phase 2: Attack Detection and Response (Rounds 9–80)

The initial drop in accuracy was recorded at Round 9 (accuracy decreased to 78.26% at Round 9). System switches to Risk Mode, with 3 helpers nodes, rather than 2. Trust scores start to distinguish between honest and malicious clients. This is because accuracy varies but it bounces back on each suspicion. Accuracy goes up to 90.45% by Round 30. The first client was banned at Round 80.

Phase 3: Stabilization (Rounds 81–200)

In the stabilization phase, the number of banned clients increases to a total of 5 at Round 169. Accuracy is stable between 94-96% and loss is low and stable between 0.13 and 0.20, with a final accuracy of 96.44% at Round 200.

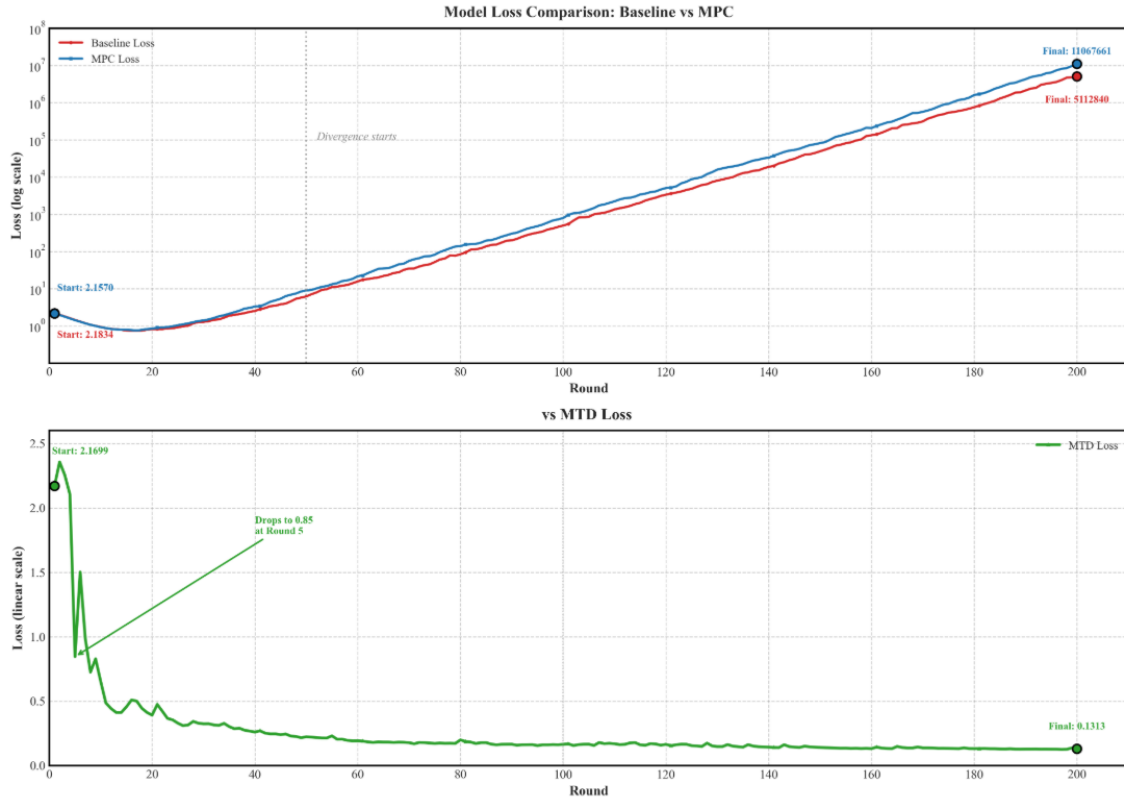


Figure 3.2: Loss Comparison: Baseline vs MPC vs MTD

Configuration	Final Loss	Loss Pattern
Baseline	5,112,839.50	Exponential growth after Round 30
MPC	11,067,661.00	Exponential growth, more severe
MTD	0.13	Stable, no growth

Table 3.2: Final Loss Comparison (Round 200)

3.2.2 Loss

After Round 30, the Baseline loss values grow exponentially, reaching 1.28 at Round 30 and more than 5 million at Round 200. This explosion means the global model is driven out of the optimum parameter space due to the amplified sign-flipping attack, where malicious updates always move the model opposite to convergence.

In comparison, the MPC setting shows even more dramatic growth. Its loss reaches 11,067,662 at Round 200, more than double the Baseline’s 5,112,840. This indicates that poisoned updates cause greater damage when secure aggregation lacks outlier detection. Exponential loss growth in MPC starts around Round 40 and accelerates after Round 60.

In contrast, the MTD approach prevents this explosion entirely. The MTD loss drops quickly after Round 5, reaches only 0.3243 at Round 30 (far below Baseline’s 1.28), remains under 0.4 throughout, fluctuates between 0.12 and 0.20 after Round 50, and ends at just 0.1313 by Round 200. This shows that MTD’s dynamic defenses neutralize the sign-flipping attack, keeping the model within the optimal parameter space and avoiding the

catastrophic divergence seen in both Baseline and MPC

3.2.3 Latency

Statistic	Baseline FL	MPC	MTD
Minimum round time	17.07 (Round 104)	17.64 (Round 98)	28.95 (Round 173)
Maximum round time	3,505.04 (Round 39)	28.44 (Round 33)	158.73 (Round 1)
Median round time	~18.5	~18.5	~30.0
Total execution time	7,430.58 seconds (~2.06 hours)	3,888.19 seconds (~1.08 hours)	6,234.06 seconds (~1.73 hours)

Table 3.3: Baseline, MPC & MTD Round Time Statistics

The strange spike at Round 39 (3,505 seconds, which is about 58 minutes) was due to a client disconnection or a client timeout event. Round times did not change during the experiment except for that anomaly, which was steady at 17–22 seconds. That stability implies that the Baseline setup has very little computational overhead per round. The MPC setup had a lower completion time than Baseline (1.08 hours versus 2.06 hours) due to the lack of the client timeout anomaly that struck Baseline in Round 39. The round times were kept constant at 17–23 seconds. The initial round of the MTD setup was far more time-consuming (158.73 seconds) since the server had to wait until every 20 clients had connected; then it was able to begin training. The average time of the subsequent rounds was about 30 seconds, which is nearly 62% longer than Baseline and MPC rounds (18.5 seconds).

3.2.4 Secure Aggregation Overhead

Metric	MPC Average Value	MTD Safe Mode (2 nodes)	MTD Risk Mode (3 nodes)
Shares wait time	~0.001 seconds	~0.000 seconds	~0.000 seconds
Node wait time (per node)	~0.006–0.010 seconds	~0.006–0.008 seconds	~0.006–0.010 seconds
Total secure aggregation time	~0.018–0.030 seconds	~0.015–0.020 seconds	~0.022–0.030 seconds
Overhead percentage	~0.1% of round time	~0.05–0.1%	~0.1%

Table 3.4: Secure Aggregation Overhead: MPC vs MTD

The MPC secure aggregation overhead was insignificant (a few percent of the overall round time), which confirms that additive secret sharing does not impose significant computational burden on the computation. Client local training and network communication occupies most of the round time (about 18 seconds round time).

The MTD secure aggregation overhead was lower at 0.1% of the round time, equivalent to the MPC configuration. The extra round time in MTD can be attributed to the computation of the trust score which involves recording the trust of all clients involved, the client selection logic which uses random sampling in Safe Mode and trust-based sorting in Risk Mode, the banning set management which verifies banned clients prior to being selected, and further logging and monitoring such as printing trust scores and chosen client IDs.

3.2.5 MTD-Results

Trust Score Evolution

Client	Selected Count	Suspicious Count	Final Trust Score	Status
Client 01	149	43	0.798	Active
Client 02	147	45	0.786	Active
Client 03	147	42	0.8	Active
Client 04	142	44	0.783	Active
Client 05	141	43	0.787	Active
Client 06	140	42	0.79	Active
Client 07	140	41	0.795	Active
Client 08	138	43	0.782	Active
Client 09	138	45	0.772	Active
Client 10	150	47	0.781	Active
Client 11	130	40	0.785	Active
Client 12	127	40	0.78	Active
Client 13	90	36	0.72	Active
Client 14	86	39	0.683	Active
Client 15	42	23	0.617	Active
Client 16	29	17	0.59	Banned (R169)
Client 17	17	10	0.588	Banned (R128)
Client 18	17	10	0.588	Banned (R80)
Client 19	16	10	0.562	Banned (R165)
Client 20	14	10	0.5	Banned (R128)

Table 3.5: Trust Score Evolution (All Clients)

Important findings related to data on trust scores:

Banned clients (5 in total): All banned clients had suspicious counts 10–17 with trust scores of less than 0.6. Client 18 was the first to be banned during Round 80. Round 128 banned clients 17 and 20. At Round 165, Client 19 was banned. Client 16 was suspended at Round 169.

Highly engaged clients (127–150 rounds): All scored between 0.772–0.800 on trust. Their level of trust was high despite the high level of suspicious counts up to 47 (Client 10) because the level of suspicion was low (suspicious/selected ≈ 0.31). Highly participating clients with low participation: Clients 13–15 have low scores (0.617–0.720) but still above

the 0.6 ban index. The reason they mistrust is mostly because of increased suspicion ratios and not because they are malicious.

Banning effectiveness: 5 out of 6 malicious clients (83%), were identified and banned. There were no honest clients who were banned (its active clients kept trust > 0.61).

Banning Timeline

The banning history indicates that malicious clients were gradually eliminated. The initial ban was during Round 80, which showed that the trust mechanism takes about 80 rounds of observation before it is sure enough to detect bad behavior. Later prohibitions were made when clients gathered enough evidence.

Suspicion Events

The main cause of suspicion was the accuracy decline between consecutive rounds, despite the overall accuracy being high (e.g., 90.45% to 89.98% at Round 28). This sensitivity enabled early warning of attacks before they were seriously degraded. The detection threshold (accuracy $<$ previous round accuracy) was effective in detecting attempts of poisoning.

Adaptive Node Selection

Mode	Active Nodes	Rounds (approx.)	Percentage
Safe Mode	2 nodes	~ 120 rounds	$\sim 60\%$
Risk Mode	3 nodes	~ 80 rounds	$\sim 40\%$

Table 3.6: Helper Node Usage Statistics

The adaptive node selection algorithm was in Safe Mode for approximately 120 rounds (60% of training) using 2 nodes, and in Risk Mode for approximately 80 rounds (40%) using 3 nodes. This demonstrates that the detection mechanism turned on Risk Mode when needed, enabling the system to run at a lower cost for the majority of the training, and increase the number of nodes when suspicious activity was detected.

3.2.6 Stability Analysis

Metric	Baseline	MPC	MTD
Accuracy range	50.67–53.32%	45.74–50.67%	94.13–96.44%
Accuracy variance	$\pm 1.33\%$	$\pm 2.47\%$	$\pm 1.16\%$
Loss trend	Increasing	Increasing	Stable
Recovery after attack	None	None	Yes (within 10–20 rounds)

Table 3.7: Performance Stability Metrics (Rounds 100–200)

MTD showed clear stability advantages in the experiment. The accuracy deviation was only $\pm 1.16\%$ in the final 100 rounds, compared to much greater deviations in Baseline and MPC. Loss was stable, showing no exponential increase, which means the model converged. Further, MTD successfully recovered in 10-20 rounds after every suspicion event and accuracy drop.

3.2.7 Computational Overhead Comparison

Configuration	Avg Round Time	Overhead vs Baseline	Secure Aggregation Overhead	Total Execution Time
Baseline	18.5 s	—	N/A	2.06 hours
MPC	18.5 s	0%	0.10%	1.08 hours
MTD	30.0 s	62%	0.10%	1.73 hours

Table 3.8: Computational Overhead Comparison

Baseline vs. MPC: Both are the same in terms of average round times (equal to approximately 18.5 seconds). The secure aggregation overhead is insignificant (0.1%). The MPC took less time (1.08 hours vs. 2.06 hours) to complete since it was not affected by the client timeout anomaly.

MTD overhead: MTD rounds are about 62% slower than Baseline/MPC (30 seconds vs. 18.5 seconds). This overhead is attributed to trust management (score computation, client sorting), dynamic client selection, banning set management and other logging.

Trade-off acceptance: The 62% round time increase is explained by the radical enhancement of the robustness (96.44% final accuracy as compared to 50.67% in Baseline). In cross-silo FL applications (e.g., hospitals, banks), this trade-off is tolerable since training takes place in hours or days.

3.2.8 Summary of Comparative Findings

Aspect	Baseline	MPC	MTD
Privacy	None	Secure aggregation	Secure aggregation
Robustness	None	None	Trust + banning
Final Accuracy	50.67%	45.74%	96.44%
Attack Mitigation	Fails	Fails	Success
Client Management	Static	Static	Dynamic (rotation + banning)
Computational Overhead	Low	Low	Moderate (+62%)
Recovery Capability	None	None	Yes

Table 3.9: Overall Comparison Summary

This section summarizes the comparative findings across all three configurations. Table 3.9 presents a structured overview of each configuration’s privacy, robustness, accuracy, and computational overhead. The results clearly show that only the MTD configuration successfully mitigates the sign-flipping attack while preserving privacy.

3.3 Discussion

3.3.1 Impact of sign-flipping on Baseline and MPC-based FL

Baseline and MPC configurations do not prevent the sign-flipping attack due to the same basic reason: they combine all client updates blindly. Additionally, the attack ($w' = -2 \times w$) is particularly effective due to the following reasons:

Sign reversal: The reversal of the sign causes the gradient to move in the opposite direction of the optimum and push the model away. In case of an honest client update is directed towards a low loss, malicious client update is directed towards a high loss.

Amplification: The malicious update is twice as powerful as an honest update, due to the factor of 2.

No filtering: The server is oblivious to anomaly detection, and averages the poisoned updates with the honest updates blindly.

3.3.2 Impact of sign-flipping on MTD-based FL

The proposed MTD-based FL performs a lot better in comparison to Baseline and MPC-based FL due to the following interrelated factors:

1. Trust-based client selection. The server can avoid the risk of choosing malicious clients by selecting clients with high scores in trust when determining the Risk Mode. The formula of trust score:

$$\text{trust} = 1.0 - 0.7 \times \left(\frac{\text{suspicious_count}}{\text{selected_count}} \right)$$

makes sure that clients that have a record of showing up during suspicious rounds are rated as less trustworthy, and thus less likely to be chosen.

2. Permanent banning. After a client has gathered enough evidence of malicious behavior ($\text{suspicious_count} \geq 10$ AND $\text{trust} < 0.6$), it is permanently removed. This will block future attacks by such a client. The dual-condition criterion avoids early exclusion of honest clients who might have occasional high loss as a result of normal training dynamics.

3. Warmup period. The 8-round warmup enables the model to set a decent accuracy threshold before the detection process starts. This eliminates false positives since the initial learning phase (where the accuracy increases fast by 24%–81%) of the model would have raised suspicion otherwise.

4. Dynamic node selection. Each round uses randomization of the helper nodes to avoid the chance of an attacker targeting particular nodes. In case of nodes being fixed, an advanced attacker could strive to intercept or spoil certain helper nodes. Randomization adds uncertainty.

5. Adaptive node count. When the attacks are suspected, using more nodes (3 vs. 2) in Risk Mode is much more secure. The attacker would have to compromise additional nodes in order to control the aggregation with 3 nodes.

6. Sensitivity to accuracy decreases. The detection mechanism will be activated in the presence of any accuracy reduction, even in cases where the absolute accuracy is high. This prompt detection enables the system to take action before major degradation is experienced.

3.3.3 Trade-offs and Limitations

Computational Overhead.

MTD rounds are about 62% slower than Baseline rounds (30s vs. 18.5s). This can be tolerated in cross-silo FL (e.g., hospitals, banks) where training is done over hours or days, and the time cost is not as high as the benefits of robustness. This overhead can be however problematic to cross-device FL when using resource-constrained devices (e.g., smartphones, IoT sensors).

Trust Parameter Tuning.

Parameter	Value	Purpose
TRUST_BAN_THRESHOLD	0.6	Minimum trust for banning
SUSPICIOUS_EVENTS_BAN	10	Minimum suspicious events for banning
TRUST_PENALTY_FACTOR	0.7	How much trust decreases per suspicious event
WARMUP_ROUNDS	8	Rounds before detection activates
STABLE_RECOVERY_ROUNDS	3	Good rounds needed to exit Risk Mode
SUSPICIOUS_ACC_THRESHOLD	0.8	Accuracy below this triggers suspicion
SUSPICIOUS_LOSS_THRESHOLD	2	Loss above this triggers suspicion

Table 3.10: MTD Trust Parameters

Thresholds are important for MTD because they are influenced by multiple factors. To detect more sophisticated adaptive attacks, lower thresholds may be required. Datasets with high variance (e.g., CIFAR-10) may need higher thresholds to avoid false positives. Moreover, if the attacker ratio is low, more suspicious events may be required for a client to be banned.

False Positive Risk.

The dual-condition banning (`suspicious_count` $\geq$ 10 AND `trust` $<$ 0.6) minimizes false positives, yet there is a possibility of false banning of honest clients with poor luck. However, this experiment did not exclude any honest clients, as the trust of all active clients remained above 0.61, highly participating clients (127–150 rounds) maintained trust above 0.77, and all banned clients had low selection counts (14–29) and high suspi-

cion ratios.

Malicious Client Evasion.

An advanced attacker could seek to avoid detection by attacking only in some rounds, using smaller perturbations (e.g. $w' = -1.1 \times w$ rather than $-2 \times w$), or planning attacks to prevent accuracy declines. Although these evasion methods might reduce detection in the short term, the trust mechanism would continue to accumulate suspicious events over time, ultimately causing banning when `suspicious_count` reaches the threshold.

3.3.4 Comparison with Related Works

The final accuracy of the MTD configuration is 96.44% when using an attacker ratio of 30% and amplified sign-flipping ($\times 2$). This compares favorably with related work:

Approach	Attacker Ratio	Final Accuracy	Key Features
FLAIR [Sharma et al.]	20-30%	$\sim 85\text{-}90\%$	continuous reputation weighting, no banning
Kianpisheh et al. (MTD-FL)	Not specified	87-96% malicious excluded	Deep RL, traffic analysis
This work (MTD)	30%	96.44%	Trust scoring perform banning

Table 3.11: Comparison with Related Work

Key differences from related work:

- FLAIR operates on continuous reputation weighting (high flip-score clients are assigned lower weights) but does not permanently block clients. It implies that there is no limit to the number of times malicious clients can keep on participating, and reputation computation must be performed continuously.
- Kianpisheh et al. achieves similar malicious exclusion rates (87%–96%) but requires deep reinforcement learning and traffic analysis, which adds significant complexity. Their solution is intended to be used in Mobile Edge Computing (MEC) and 6G wireless models.

The proposed MTD mechanism achieves comparable or superior robustness with a more interpretable trust scoring mechanism that does not necessitate any reinforcement learning, traffic analysis, or access to individual model updates.

3.3.5 Implications for Federated Learning Security

The experimental findings have a number of implications on practical federated learning deployments:

1. Privacy is not robustness. MPC offered powerful privacy, but no robustness. Privacy-sensitive deployments (e.g., medical, financial) should use extra defenses on top of secure aggregation.

2. Early detection is critical. The fact that MTD is sensitive to accuracy (even slight ones) allowed it to be detected early enough before considerable degradation would have taken place. Deployments must keep track of trends in accuracy, but not absolute values.

3. Permanent banning is effective. Permanent banning, unlike continuous reputation weighting, eliminates bad clients completely. Once banned, the system does not have to calculate trust scores of banned clients or worry about their participation in the future.

4. Warmup periods reduce false positives. The 8-round warm up allowed the avoidance of false positive in the initial learning phase. Deployments should calibrate warmup duration based on expected convergence time.

5. Cross-silo FL is the appropriate use case. The 62% round time overhead is tolerable in cross-silo FL (organizations with dedicated resources) but can be challenging in cross-device FL (resource-constrained devices).

3.4 Summary

This chapter demonstrated a detailed analysis of three federated learning configurations under a sign-flipping poisoning attack (30% malicious clients, amplification factor of 2). The Baseline configuration achieved a best accuracy of 78.80% before dropping to 50.67% by Round 200, and an exponentially increasing loss of more than 5 million. The MPC configuration also suffered degradation to 45.74% with a loss of 11 million, showing that privacy protection is not sufficient for robustness.

Our proposed MTD configuration, on the other hand, reached 96.44% accuracy with a stable loss of 0.13, banning 5 out of 6 malicious clients (83%) while not banning any honest client. MTD consisted of three phases: a warm-up phase (Rounds 1-8), attack detection and response (Rounds 9-80), and convergence (Rounds 81-200). Despite 62% longer rounds in the MTD configuration (30s vs. 18.5s), the overhead for the secure aggregation of the clients was negligible at 0.1% for all configurations. In all, the findings show that the trust scoring system is able to successfully discriminate between honest and malicious participants, and that only the MTD configuration is resilient to poisoning attack.

The next chapter concludes the thesis with a summary of work, limitations and future work.

Chapter 4

Conclusions and Perspectives

This chapter summarizes the research presented in this thesis, highlights the main contributions, discusses limitations of the current work, and suggests directions for future research.

4.1 Summary of the Work

The research presented in this thesis studied the vulnerability of Federated Learning to sign-flipping poisoning attacks and tested three configurations with different levels of defense: a Baseline (no defense), an MPC-based secure aggregation configuration, and a proposed MTD (Moving Target Defense) configuration. The three configurations were tested on the MNIST dataset over 200 communication rounds, with 30% malicious clients applying an amplified sign-flipping attack ($\delta' = -2\delta$). Our experiments show that unprotected Federated Averaging is not resilient against a moderate number of malicious clients, and that secure aggregation (MPC) only offers privacy, not resilience. The proposed MTD architecture, which includes MPC-based secure aggregation, client scoring via trust-based reputation, permanent banning and dynamic client selection, and adaptive rotation of the helper nodes, was able to successfully protect against the attack with 96.44% final accuracy.

4.2 Limitations

Although the results are encouraging, there are still limitations to consider:

1. **Single run per configuration:**
Only a single run was performed. Repeated runs would allow statistical confidence intervals to be calculated and would reduce the effects of random variation.
2. **Single attack type:**
Only the sign-flipping attack with a constant magnitude (2.0) was tested. Adaptive attacks that try to evade detection by decreasing the strength or attacking less frequently were not evaluated.

3. IID data distribution:

We used IID MNIST data. In real-world FL, the data is often non-IID, which could influence the convergence rate and the detection thresholds based on accuracy.

4. Simple model architecture:

A two-layer MLP was used. The potential of the MTD defense with other models such as CNNs or transformers and other datasets such as CIFAR-10 has yet to be explored.

5. One malicious client evaded banning:

One client was not banned by Round 200. This indicates that clients with a lower selection probability could need to be observed for a longer period to trigger banning.

4.3 Conclusions

This thesis demonstrates that privacy and robustness can be achieved simultaneously in federated learning without inspecting individual client updates. The MPC configuration showed that achieving robustness solely through privacy-preserving measures is insufficient, as it was slightly worse under attack than the unprotected Baseline FL (45.74% vs. 50.67%).

The MTD configuration addressed this issue by using secure aggregation, trust-based scoring and permanent banning, client rotation, and dynamic node selection. Our findings indicate that monitoring only the global model metrics is sufficient to detect and ban malicious clients. The proposed MTD banned 5 of the 6 malicious clients (83%) without banning any honest client, and reached an accuracy of 96.44%, compared to 50.67% for the Baseline FL. Although MTD rounds were 62% longer than the other configurations, this overhead is tolerable in cross-silo FL deployments such as hospitals and financial institutions. Overall, our proposed approach demonstrates that a privacy-preserving, inspection-free mechanism to defend against poisoning attacks is practical and effective.

4.4 Future Perspectives

This thesis suggests a number of future research directions. First, the proposed thresholds were constants, and a dynamic threshold mechanism that considers model variance would reduce the risk of false positives, especially for more difficult datasets (CIFAR-10 and FEMNIST). It would also be beneficial to evaluate the MTD under adaptive attacks where the attacking clients know about the trust mechanism and attempt to evade detection by injecting small noise and sporadically attacking.

Beyond the current work, testing the MTD with non-IID data would be more realistic for FL where the data is not identically distributed. Increasing the number of clients to several hundred would also ensure the scalability of the trust scoring and banning. Complementing the MTD with Byzantine-robust aggregators such as Trimmed Mean or FLTrust could also provide an additional layer of security, as it pairs update monitoring

with update inspection in non-private FL. Finally, repeating each experiment several times with different random seeds would increase the reliability of the results and reduce the influence of random events such as client timeouts.

4.5 Code Availability

The complete implementation code for this thesis is publicly available on GitHub at the following link:

[https://github.com/husnainali98/
Thesis-FL-MTD-Attack-Analysis-and-Defense-Optimization](https://github.com/husnainali98/Thesis-FL-MTD-Attack-Analysis-and-Defense-Optimization)

References

- [1] S. Alawadi et al. “MPCFL: Towards Multi-party Computation for Secure Federated Learning Aggregation”. In: *ACM UCC*. 2023.
- [2] Gilad Baruch, Moran Baruch, and Yoav Goldberg. “A Little Is Enough: Circumventing Defenses For Distributed Learning”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2019.
- [3] Daniel J Beutel et al. *Flower: A Friendly Federated Learning Research Framework*. 2020.
- [4] Peva Blanchard et al. “Machine Learning with Adversaries: Byzantine Tolerant Gradient Descent”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2017.
- [5] Keith Bonawitz et al. “Practical Secure Aggregation for Privacy-Preserving Machine Learning”. In: *Proceedings of ACM CCS*. 2017.
- [6] Keith Bonawitz et al. “Towards Federated Learning at Scale: System Design”. In: *Proceedings of MLSys* (2019).
- [7] Xiaoyu Cao et al. “FLTrust: Byzantine-robust Federated Learning via Trust Bootstrapping”. In: *NDSS*. 2021.
- [8] Minghong Fang et al. “Local Model Poisoning Attacks to Byzantine-Robust Federated Learning”. In: *USENIX Security Symposium*. 2020.
- [9] Inc. Grand View Research. *Federated Learning Market Size, Share & Trends Analysis Report By Application (Industrial Internet of Things, Drug Discovery, Risk Management), By Organization Size, By Industry Vertical, By Region, And Segment Forecasts, 2025 - 2030*. Industry Report GVR-4-68040-162-2. Grand View Research, 2025.
- [10] Charles R Harris, K Jarrod Millman, Stéfan J van der Walt, et al. “Array programming with NumPy”. In: *Nature* 585 (2020), pp. 357–362.
- [11] S. Kianpisheh, C. Benzaid, and T. Taleb. “Multi-Model based Federated Learning Against Model Poisoning Attack: A Deep Learning Based Model Selection for MEC Systems”. In: *IEEE Globecom*. 2024.
- [12] S. Kianpisheh and T. Taleb. “Deep Learning based Moving Target Defence for Federated Learning against Poisoning Attack in MEC Systems with a 6G Wireless Model”. In: *arXiv preprint arXiv:2509.10914* (2025).

- [13] P. Mai, R. Yan, and Y. Pang. “RFLPA: A Robust Federated Learning Framework against Poisoning Attacks with Secure Aggregation”. In: *Advances in Neural Information Processing Systems (NeurIPS)* 37 (2024).
- [14] Brendan McMahan et al. “Communication-Efficient Learning of Deep Networks from Decentralized Data”. In: *Proceedings of AISTATS*. 2017.
- [15] Adam Paszke et al. “PyTorch: An Imperative Style, High-Performance Deep Learning Library”. In: *Advances in Neural Information Processing Systems*. Vol. 32. 2019.
- [16] A. Sharma et al. “FLAIR: Defense against Model Poisoning Attack in Federated Learning”. In: *ACM ASIA CCS*. 2023.
- [17] Anshuman Sharma and Ningrinla Marchang. “Probabilistic Sign Flipping Attack in Federated Learning”. In: *IEEE ICCCNT*. 2024.
- [18] Virat Shejwalkar et al. “Back to the Drawing Board: A Critical Evaluation of Poisoning Attacks on Production Federated Learning”. In: *IEEE Symposium on Security and Privacy*. 2022.
- [19] TorchVision Contributors. *TorchVision: PyTorch’s Computer Vision Library*. <https://github.com/pytorch/vision>. 2016.
- [20] Yi Xie, Minghong Fang, and Neil Zhenqiang Gong. “Model Poisoning Attacks to Federated Learning via Multi-Round Consistency”. In: *IEEE/CVF CVPR*. 2025.
- [21] Dong Yin et al. “Byzantine-Robust Distributed Learning: Towards Optimal Statistical Rates”. In: *Proceedings of ICML*. 2018.

Appendix A

Implementation Code Snippets

A.1 Introduction

This appendix includes some code snippets from the implementation of the three federated learning configurations evaluated in this thesis: Baseline, MPC-based, and MTD-based FL. The snippets are focused on the key functionalities: the model architecture, the sign-flipping attack, the additive secret sharing mechanism, and the trust scoring and banning logic of the MTD.

A.2 Model Architecture

The exact two-layer MLP was used in all three configurations. The model takes a flattened 28×28 MNIST image as input, passes it through a hidden layer of 64 units with ReLU activation, and outputs logits for 10 digit classes.

```
1 class Net(nn.Module):
2     def __init__(self):
3         super().__init__()
4         self.fc1 = nn.Linear(28 * 28, 64)
5         self.fc2 = nn.Linear(64, 10)
6
7     def forward(self, x):
8         x = x.view(-1, 28 * 28)
9         x = F.relu(self.fc1(x))
10        return self.fc2(x)
```

A.3 Sign-Flipping Attack

The sign-flipping attack takes place on the client side after each round of local training and before the update is sent to the server. The attack flips the sign of the gradient and amplifies it by 2.0 times, thus making the global model diverge. The attack is applied the

same way in all three configurations.

```
1 delta = new_nds[i].astype(np.float64) - global_nds[i].astype(np.float64)
2 # -----
3 # MODEL POISONING ATTACK(flip)
4 # -----
5 if self.is_malicious:
6     if self.attack_type == "scale":
7         delta = delta * self.attack_strength
8     elif self.attack_type == "flip":
9         delta = -delta * self.attack_strength
```

A.4 Additive Secret Sharing

The `split_additive_multi` function takes a client's model update and splits it into one share for the server and k shares for the helper nodes such that the sum of all shares will be exactly the same as the original model update. This is the key function of the secure aggregation in the MPC and MTD configurations.

```
1 def split_additive_multi(x: np.ndarray, num_node_shares: int, rng: np.
   random.Generator | None = None):
2     if rng is None:
3         rng = np.random.default_rng()
4
5     # Random share for server
6     server_share = rng.normal(loc=0.0, scale=1.0, size=x.shape).astype(
       np.float64)
7
8     # Special case: only one helper node
9     if num_node_shares == 1:
10        node_share = (x.astype(np.float64) - server_share).astype(np.
       float64)
11        return server_share, [node_share]
12
13    # Create random shares for first k-1 nodes
14    node_shares = []
15    running_sum = server_share.copy()
16
17    for _ in range(num_node_shares - 1):
18        sh = rng.normal(loc=0.0, scale=1.0, size=x.shape).astype(np.
       float64)
19        node_shares.append(sh)
20        running_sum = running_sum + sh
21
22    # Last node share makes the sum exact
23    last_share = (x.astype(np.float64) - running_sum).astype(np.float64)
24    node_shares.append(last_share)
25
26    return server_share, node_shares
```

A.5 MTD Trust Score Computation

The trust of each client is updated after each round where suspicious activity is detected. The greater the number of suspicious rounds a client is part of, the lower its trust. The trust is reduced by a penalty factor and will never fall below a threshold.

```

1 def recompute_trust(cid: str):
2     ensure_client_exists(cid)
3
4     selected = client_selected_count[cid]
5     suspicious = client_suspicious_count[cid]
6
7     if selected <= 0:
8         client_trust_score[cid] = 1.0
9         return
10
11     suspicious_ratio = suspicious / float(selected)
12     trust = 1.0 - TRUST_PENALTY_FACTOR * suspicious_ratio
13     client_trust_score[cid] = max(MIN_TRUST_SCORE, trust)

```

A.6 MTD Banning Mechanism

Once each suspicious round ends, the server increases the number of suspicious events for all clients in that round, and updates their trust scores. A client is permanently banned only if it meets both conditions, exceeding the suspicious event threshold and falling below the trust score threshold, ensuring honest clients are not excluded based on a single suspicious round.

```

1 for cid in selected_ids:
2     client_suspicious_count[cid] += 1
3     recompute_trust(cid)
4
5     # Check if client should be banned (MORE REALISTIC: both conditions)
6     if (client_suspicious_count[cid] >= SUSPICIOUS_EVENTS_BAN and
7         client_trust_score[cid] < TRUST_BAN_THRESHOLD):
8         if cid not in banned_clients:
9             banned_clients.add(cid)

```

A.7 MTD Adaptive Node Selection

The number of active MPC helper nodes are adjusted dynamically depending on the risk state. In Safe Mode, a lower fraction of the pool is used and during Risk Mode, a higher fraction. The nodes are chosen randomly from the node pool every round to ensure no fixed node is always targeted.

```

1 def compute_active_node_count(total_nodes: int, risky: bool) -> int:

```

```
2     if total_nodes <= 0:
3         raise RuntimeError("No MPC nodes configured.")
4
5     if risky:
6         count = int(ceil(RISK_NODE_RATIO * total_nodes))
7         count = max(MIN_RISK_NODES, count)
8     else:
9         count = int(ceil(SAFE_NODE_RATIO * total_nodes))
10        count = max(MIN_SAFE_NODES, count)
11
12    return min(total_nodes, count)
```